Artificial Intelligence Laboratory

Lab Experiment 5 — Study of PROLOG

6th Semester · B.Tech CSE · NIT Srinagar

Student Name		**Enrollment No.**	
Batch		**Date**	

Instructions: Answer all 4 problems. For each problem: (a) write the PROLOG facts and rules, (b) show a query trace (how PROLOG resolves it step by step), (c) write the queries and their expected outputs, and (d) answer the analysis questions. Remember: facts and rules end with `.` (period). Variables start with **Uppercase**.

Quick Reference — PROLOG Syntax

Concept	Syntax	Meaning
Fact	`likes(tom, pizza).`	Tom likes pizza (always true)
Rule	`head :- body.`	head is true IF body is true
Variable	`X, Y, Name`	Uppercase = variable
Constant	`tom, 42, "hello"`	Lowercase = constant
Conjunction	`goal1, goal2`	AND (both must succeed)
Disjunction	`goal1 ; goal2`	OR (either can succeed)
Negation	`\+ goal`	NOT (goal fails)
Arithmetic	`X is 3 + 4`	Evaluate expression
Unification	`X = 5`	Bind X to 5
Cut	`!`	Stop backtracking
`findall`	`findall(X,Goal,List)`	Collect all solutions
List pattern	`[H|T]`	Head and Tail of list

1

Scenario: Student–Course–Faculty Relationships

A university database stores information about students, courses, and faculty members. Build a PROLOG knowledge base with the following information and answer queries about it.

Students: Ahsan (CS, 3rd year), Sara (CS, 2nd year), Ali (EE, 3rd year), Zara (CS, 2nd year)

Courses: AI (taught by Dr. Shaikh, CS dept), Networks (taught by Dr. Khan, CS dept), Circuits (taught by Dr. Mir, EE dept)

Enrollment: Ahsan and Sara enrolled in AI; Sara and Zara in Networks; Ali enrolled in Circuits.

Tricky part: Write rules for:

- `classmate(X, Y)` — two different students in the same course
- `dept_student(D, S)` — student S belongs to department D
- `teaches_dept(F, S)` — faculty F teaches student S
- `busy_student(S)` — student enrolled in more than one course

Syntax hint: To say "Ahsan is in CS department, 3rd year": `student(ahsan, cs, 3).`

```
enrolled(ahsan, ai).
course(ai, dr_shaikh, cs).
```

Tasks:

1. Write all **facts** for students, courses, and enrollment. **[4 Marks]**
2. Write the four **rules** listed above. **[6 Marks]**
3. Write queries and expected outputs for: **[6 Marks]**
   - Who are Sara's classmates?
   - Which faculty members teach Ahsan?
   - Which students are busy (multiple courses)?
   - Find all CS students

4. **Trace:** How does PROLOG resolve `?- classmate(sara, X)`? Show each unification step. **[4 Marks]**

(a) Facts:

(b) Rules:

(c) Queries and expected outputs:

Query	Expected Output
`?- classmate(sara, X).`	
`?- teaches_dept(F, ahsan).`	
`?- busy_student(S).`	
`?- dept_student(cs, S).`	

(d) Resolution trace for `?- classmate(sara, X)`:

1

Scenario: Custom List Library in PROLOG

In PROLOG, lists are written as `[1,2,3]` or using the head-tail pattern `[H|T]`. Write recursive PROLOG predicates to implement a small list processing library from scratch.

Implement the following predicates:

1. `my_last(List, X)` — X is the last element of List
2. `my_reverse(List, Rev)` — Rev is List reversed
3. `my_max(List, Max)` — Max is the largest element
4. `my_flatten(Nested, Flat)` — flatten a nested list
5. `count_occurrences(X, List, N)` — count how many times X appears

Tricky part: Each predicate must work *recursively*. You cannot use built-in predicates like `last/2` or `reverse/2`. The base case is always the empty list `[]`.

List pattern hint:
`[H|T]` matches a list where `H` = first element, `T` = rest of list
`[1,2,3]` = `[1|[2|[3|[]]]]` in PROLOG's internal representation
Accumulator pattern: Use a helper predicate with an extra argument to build results.

Tasks:

1. Write `my_last/2` and `my_reverse/2` with base + recursive cases. **[6 Marks]**
2. Write `my_max/2` (hint: compare head with max of tail). **[4 Marks]**
3. Write `count_occurrences/3` using pattern matching. **[4 Marks]**
4. Show query outputs for: **[3 Marks]**
   - `?- my_last([1,2,3,4], X).`
   - `?- my_reverse([a,b,c], R).`
   - `?- count_occurrences(3, [1,3,2,3,4,3], N).`
5. **Trace:** Unroll `my_last([a,b,c], X)` step by step. **[3 Marks]**

(a) `my_last/2` **and** `my_reverse/2`**:**

(b) `my_max/2`**:**

(c) `count_occurrences/3`**:**

(d) Query outputs:

`?- my_last([1,2,3,4], X).` X = _____

`?- my_reverse([a,b,c], R).` R = _____

`?- count_occurrences(3,[1,3,2,3,4,3],N).` N = _____

(e) Trace for `my_last([a,b,c], X)`**:**

1

Scenario: Animal Classification Expert System

Build a PROLOG expert system that classifies animals based on their **physical characteristics** (has_hair, has_feathers, lays_eggs, has_gills, warm_blooded, has_legs, has_backbone).

Classification rules:

- **Mammal**: has hair AND warm-blooded
- **Bird**: has feathers AND lays eggs AND has backbone
- **Reptile**: lays eggs AND has backbone AND cold-blooded (NOT warm-blooded)
- **Fish**: has gills AND has backbone AND lays eggs
- **Amphibian**: has backbone AND lays eggs AND (can live in water AND on land)

Animals to classify: dog, eagle, snake, salmon, frog

Tricky part:

- Use `\+` (not) for cold-blooded (absence of warm_blooded)
- Write a `classify(Animal, Class)` rule combining all cases
- Use `findall` to get a complete classification report

Negation hint:

```
cold_blooded(X) :- \+ warm_blooded(X).
```

This says X is cold-blooded if we CANNOT prove X is warm-blooded.

findall hint:

`findall(A-C, classify(A,C), Report)` collects all Animal-Class pairs.

Tasks:

1. Write facts for all 5 animals (their physical properties). **[5 Marks]**
2. Write the 5 classification rules. **[6 Marks]**
3. Write `classify(Animal, Class)` that unifies all 5 rules. **[3 Marks]**
4. Write a query using `findall` to classify all animals and show output.

[4 Marks]

5. **Tricky:** Add a new animal "platypus" (has hair, lays eggs, warm-blooded). Which class does it fall into? Does PROLOG classify it correctly? What does this reveal about rule-based systems? [2 Marks]

(a) Facts for 5 animals:

(b) Classification rules:

(c) classify/2 rule:

(d) findall query and output:

Query: _____

Expected output:

(e) Platypus analysis:

What class does PROLOG assign? _____

Is this biologically correct? _____ Why?

1

Scenario: Towers of Hanoi in PROLOG

The Towers of Hanoi puzzle has 3 pegs: **left**, **middle**, **right**. N disks are stacked on the left peg (smallest on top). Move all disks to the right peg following the rules:

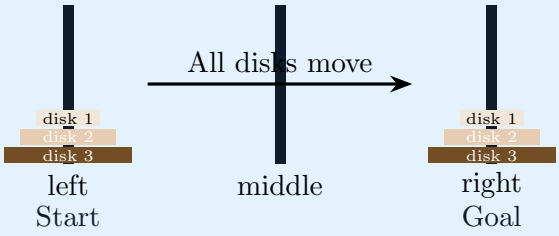
1. Only one disk can be moved at a time
2. A larger disk cannot be placed on a smaller disk
3. All three pegs can be used

PROLOG is perfect for this: The recursive structure of the problem maps directly to recursive PROLOG rules. PROLOG's backtracking handles all the steps automatically.

Algorithm:

- Base case: Move 1 disk from `From` to `To` directly
- Recursive: Move $N-1$ disks to `Mid`, then move disk N to `To`, then move $N-1$ disks from `Mid` to `To`

Expected output for N=3: Move disk 1 from left to right, move disk 2 from left to middle, ... (7 total moves)



Tasks:

1. Write the PROLOG `hanoi(N, From, To, Via)` predicate with base case and recursive case. **[8 Marks]**
2. Run the query `?- hanoi(3, left, right, middle).` and write all 7 move outputs. **[4 Marks]**
3. **Mathematical analysis:** For N disks, how many moves are needed? Write the recurrence relation and solve it. **[4 Marks]**

4. **Tricky:** PROLOG's recursion for Hanoi is clean and elegant. Could you write the same algorithm in Python without a stack? Why does PROLOG's backtracking engine make recursive solutions natural? **[4 Marks]**

(a) PROLOG `hanoi/4` predicate:

(b) All 7 moves for N=3 (fill in):

Move	Action
1	Move disk 1 from left to ___
2	
3	
4	
5	
6	
7	Move disk 1 from ___ to right

(c) Recurrence relation:

$T(1) =$ _____ $T(N) =$ _____ $T(N) =$ _____ (closed form)

(d) Comparison with Python: